

輔仁大學-資訊工程所

Graduate Optional course

Saturday 8:30AM to 11:20AM

周賜福

Oct 2020

**Dept. of Computer Science and
Information Engineering.**

Fu Jen Catholic University,

Multicore System and parallel Programming

多核心系統與平行化程式設計

Using Multithreaded programming.

Oct 5th, 2020.

Fu Jen Catholic University.

Hsin Chuang, 242

Today's content of the class

Exploring what is process and Multithread?

What are the differences and the advantage of a thread?

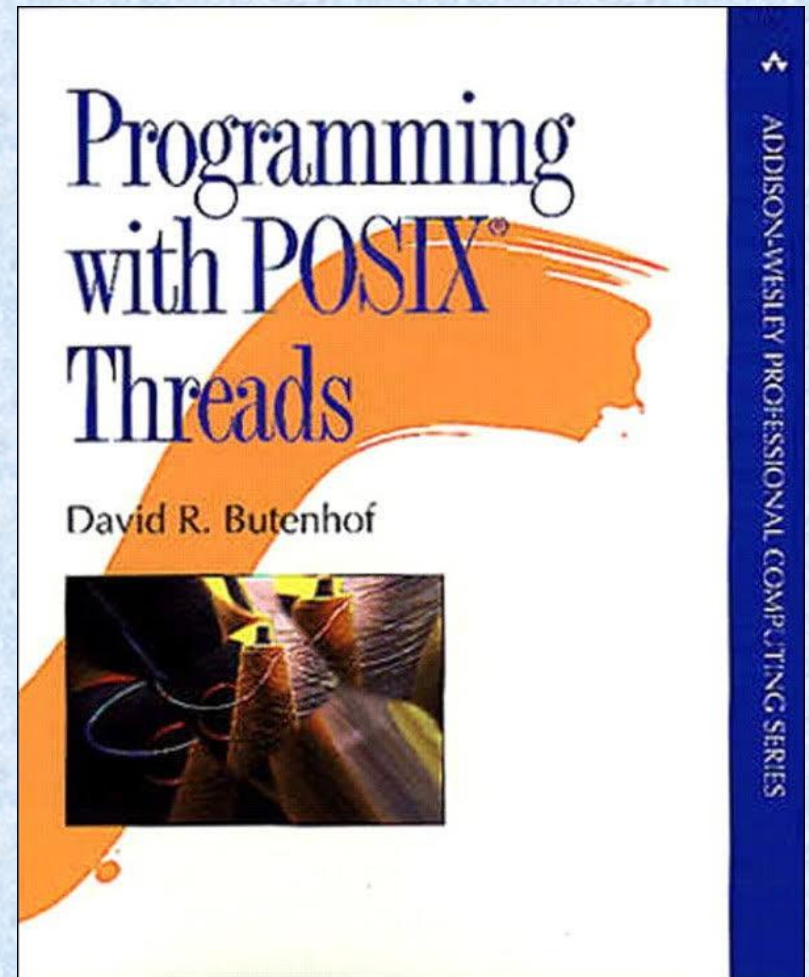
Learn the basics of programming in Pthread.

Write some examples and understand the basic concepts of a thread?

Good Link for a detailed explanation using and writing Pthread programs

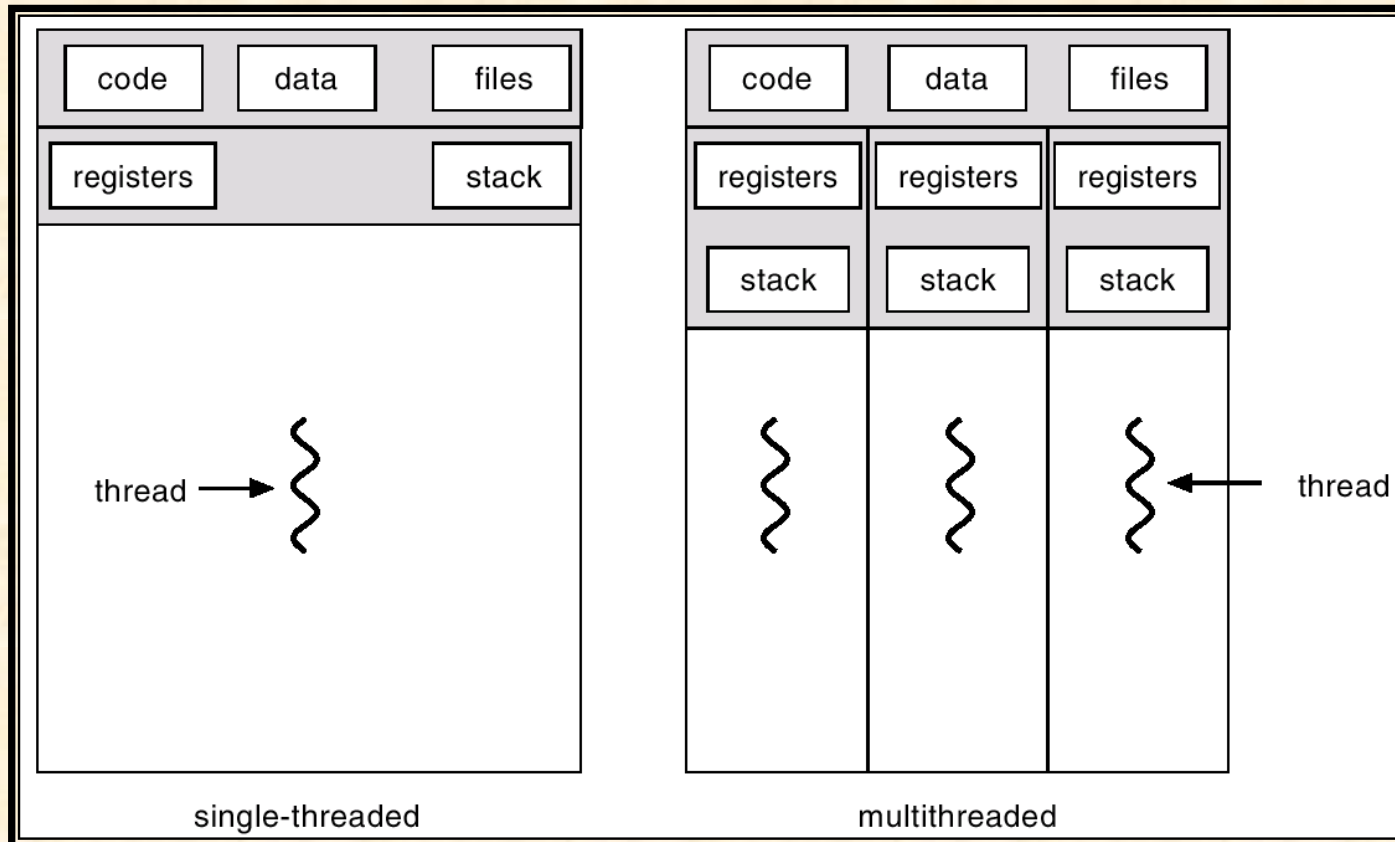
[Good Link to understand what is thread?](#)

Good Text book for detailed understanding of Multi-threaded program



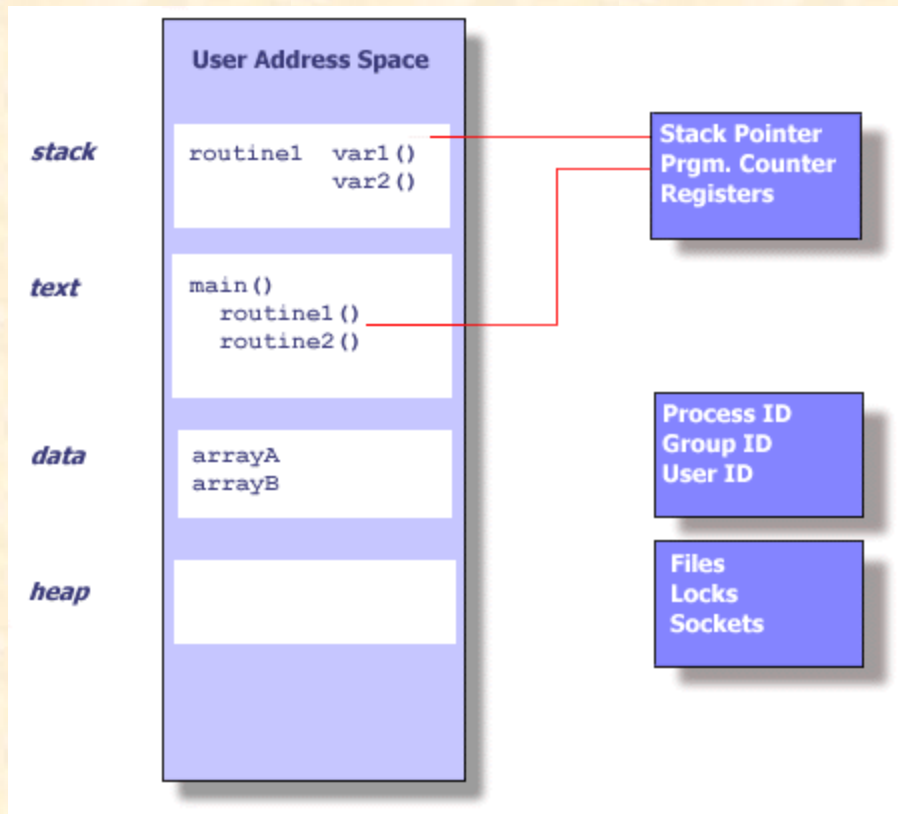
Single and Multithreaded Processes

單執行緒和多執行緒的行程

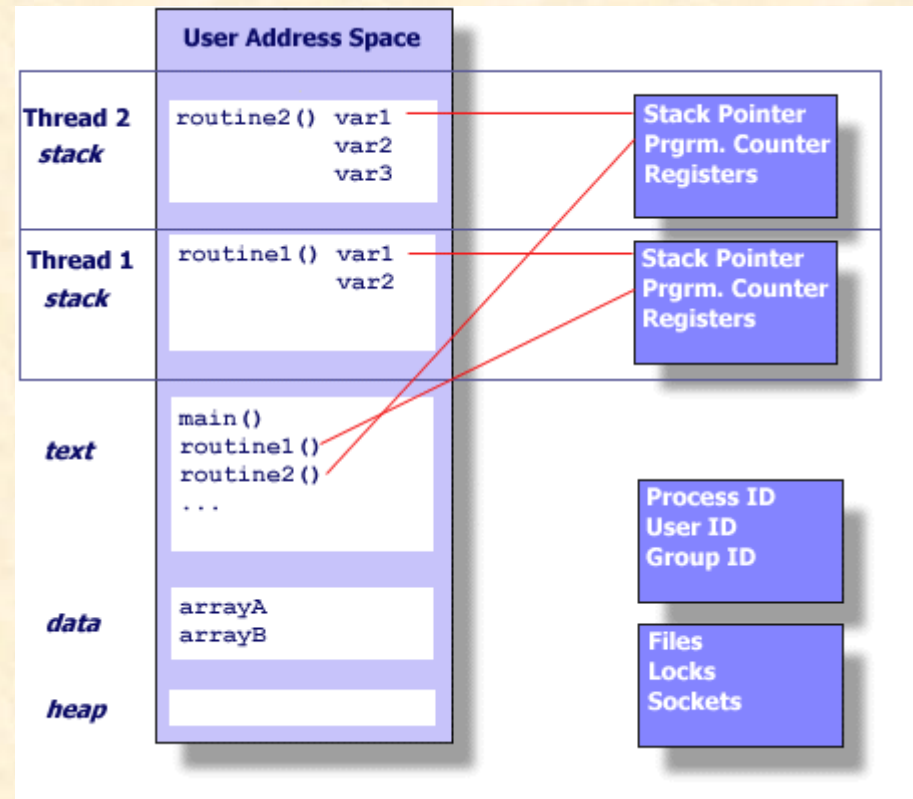


Process vs thread.

Threads use and exist within these process resources, yet are able to be scheduled by the operating system and run as independent entities largely because they duplicate only the bare essential resources that enable them to exist as executable code.



UNIX PROCESS



THREADS WITHIN A UNIX PROCESS

Process vs thread.

Process: It is called **heavyweight** process and has single thread of control. (**Heavyweight** – 重量級行程)

Process based models assume that all data associated with a process is private, by default unless otherwise specified.
(Unix shmget and shmat.)

The overheads associated with enforcing protection domains make processes less suitable for parallel programming.

This independent flow of control is accomplished because a thread maintains its own:

- Stack pointer
- Registers
- Scheduling properties (such as policy or priority)
- Set of pending and blocked signals
- Thread specific data.

If process has multiple threads of control, it can do more than one task at a time.

Process vs thread.

So, in summary, in the UNIX environment a thread:Exists within a process and uses the process resources.

Has its own independent flow of control as long as its parent process exists and the OS supports it.

Duplicates only the essential resources it needs to be independently schedulable.

May share the process resources with other threads that act equally independently (and dependently).

Dies if the parent process dies - or something similar.

Is "**lightweight**" because most of the overhead has already been accomplished through the creation of its process.

Why do we need more threads?

Sometime one application may have to perform several similar tasks.

Example: 1. Take a busy web server. It has to respond to many clients such as web pages, images, sound, and so on.

2. A word processor may have a thread for displaying graphics, another thread for reading a keystrokes from the user, and a third thread for performing spelling check and so on.

Instead of creating many processes to handle, it can create many threads and handle the requests of each client.

One server when it receives a request, it creates several processes to handle the requests or similar tasks.

Instead, the server creates several threads to handle the similar tasks. Since the thread is lightweight, the overhead would be less.

There are several benefits in using threads rather than processes.

Look at what are benefits of having thread rather than processes.

Benefits 利益 of thread

- Responsiveness: 應答 Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operations
- Resource Sharing: (資訊分享) Threads share the memory and the resources of the process to which they belong.
- Economy: (經濟) Allocating of processes is very costly and the overhead is more than the thread.
- Utilization of MP Architectures: (使用多處理器架構) The benefits of multithreading can be greatly increased in a multiprocessor architecture, where each thread may be running in parallel on a different processor.

Parallel Programming

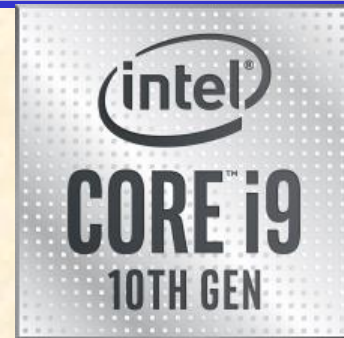
平行程式設計

Kernel Level

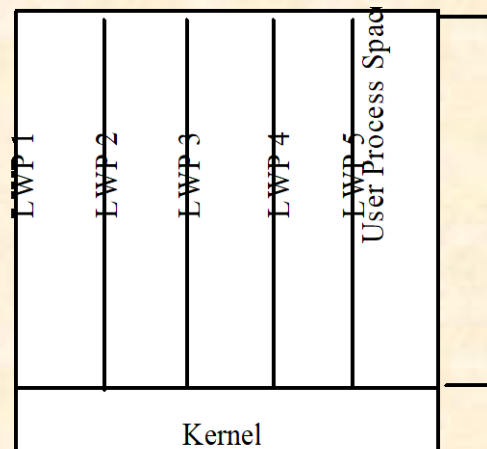
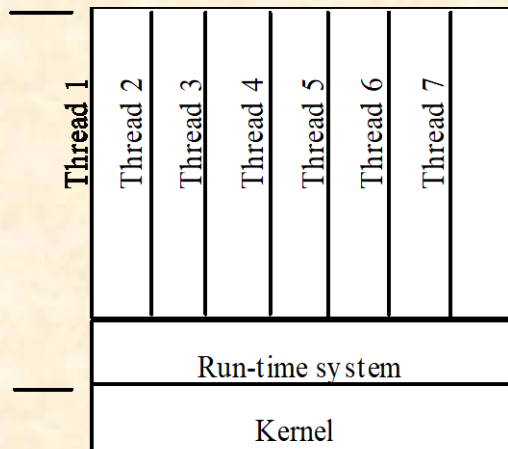
Mach, NT, Solaris, OSF, Linux threads

Use system calls to create, spawn and synchronize

Needed to run threads on multiprocessor SMP systems



Kernel Level vs User Level threads



Intel® Core™ i9-10900T 處理器

- 20 MB Intel® Smart Cache 快取記憶體
 - 10 核心
 - 20 執行緒
 - 4.60 GHz 最大超頻
 - T - Power-optimized lifestyle
 - 10th Generation
- 加入比較

User Level threads.

OS kernel does not know of the existence of threads.

A runtime system is responsible for creating and managing user level threads.

Parallel Programming

平行程式設計

User Level threads.

The runtime library catches all thread calls to system. If the call leads to blocking, the runtime system switching in a new thread for execution.

- o Can be built on existing non-threaded OS
- o More efficient since no system calls are needed for creation and management of threads
- o Process specific resource management can be implemented

How to detect if a thread's call blocks or not

Page faults?

No per thread timer

Kernel Level threads

OS knows of threads. Thread creation and management involves system calls.

Less efficient since every call involves the kernel

Cannot support high degree of parallelism but can be used in SMP systems

Also, easier to deal with page faults, run-away threads

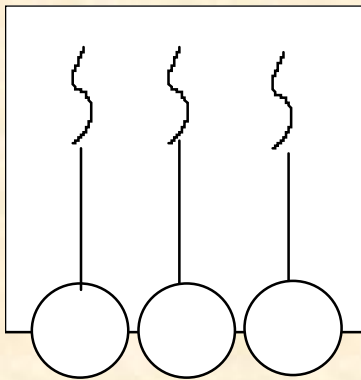
Possibility of switching threads on remote memory accesses

Thread Execution Models

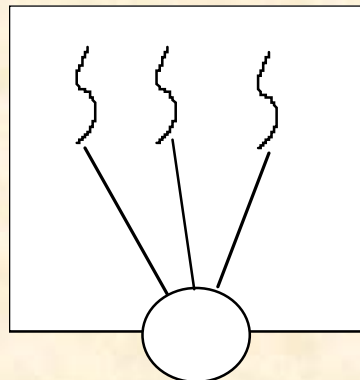
when we have kernel level and user level threads (particularly in Solaris)

Should we assign one kernel level thread to each user level thread, or bind multiple user level threads to a single kernel level thread?

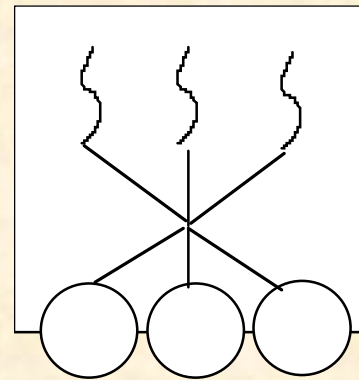
One-to-One



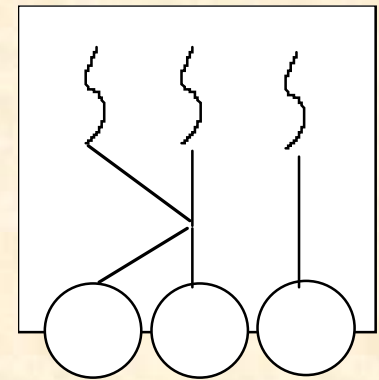
Many-to-One



Many-to-Many



Combination



Multithreaded Programming

What is needed in multithreaded model

In any concurrent programming model we need:

Mechanisms for the creation and management of concurrent activities

- Creating or Forking threads
- Killing threads
- Suspending and Resuming threads
- Scheduling -- priorities
- Managing resource requirements
 - Stack size
 - Stack address
- Binding of an activity to the “processing” thread
 - Passing code (or executable) to the created thread
 - Passing parameters to the code being executed

Need to understand

- when a thread is created,
- when a thread actually starts execution,
- when a thread terminates naturally,
 - o what happens on termination
 - o what happens if a thread is terminated abnormally resources
 - o status of threads awaiting a response
 - o issues related to performance

Different thread libraries, kernels and languages have different semantics with these.

Mechanisms for coordination among the concurrent activities (threads and MPI)

- Waiting for results from other threads

- Sharing a variable

- Communicating data or control

- Waiting for certain events to happen

General techniques

- Message passing model

 - Sending thread is blocked until the message is received

 - Receiving thread is blocked until a message is received

- Mutexes and semaphores (for threads)

 - Acquire a lock before proceeding

 - Used to guarantee a single thread in “critical section”

 - Counting semaphores can be used to manage resources.

Barrier synchronization

Threads must wait for other threads to reach a barrier
and a single thread continues beyond the barrier

Monitors

Shared data is modified by a monitor
Threads must enter the monitor to access shared data
Only one thread can enter the monitor at a time

Conditions and Events

Threads can wait for a specific event or condition
Threads are blocked until the event/condition is signaled

Multithreading implies a Shared Memory Model

No IPC --communication among threads achieved using
shared variables

We will not use send-receive to communicate.

We will use mutexes, semaphores, barriers, condition variables

An Overview of Pthreads

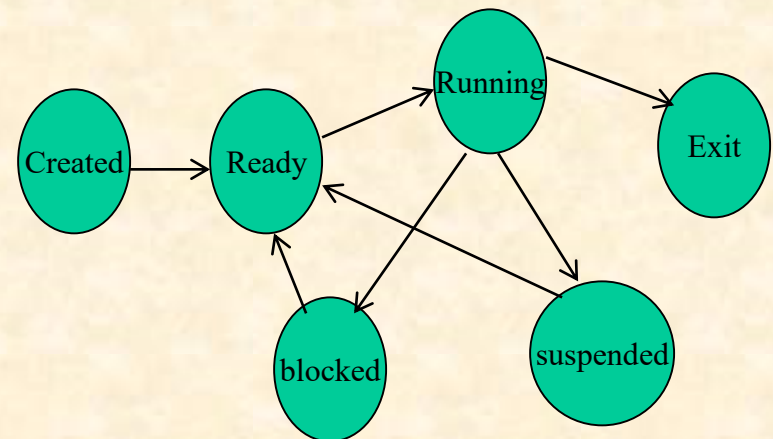
Pthreads are part of the POSIX (Portable Operating System Interface) standard developed by the IEEE.

Pthreads defines the standard API's for developing multithreaded applications on POSIX compliant systems.

There are some variations on which Pthread standard is supported on different systems.

Most likely, you will see Draft 7 (we will use this in this presentation), some more recently implementations have used Draft 10.

The Life Cycle of a Thread



Now let us enter into programming concept of User level threads

Posix Thread types and the system calls used for programs

pthread_t	thread identifier
pthread_create	creating a thread
pthread_join	wait for the thread
pthread_exit	pthread exits.
pthread_mutex_t	mutex
pthread_cond_t	condition variable
pthread_key_t	“access key” for thread-specific data
pthread_attr_t	thread attribute object
pthread_mutexattr_t	mutex attributes object
pthread_condattr_t	condition variable attributes object
pthread_once_t	one time initialization control context

Creating A Thread.

```
int pthread_create (pthread_t *thread, const pthread_attr_t *attr,  
void * (*start_routine) (void *), void * arg);
```

Thread ID is returned in the first argument. Must be declared as a special data type
pthread_t

The second argument defines thread attributes. NULL means default values
We can define our own attributes to control stack size, priority, etc.

The third argument is the function to be executed by the thread. Note the function is of type
Void (cannot return a value)

The last argument is a single parameter to the function being executed by the thread. Only one
parameter can be passed to a thread

Any thread can create other threads -- and any thread can kill other threads!

Creating A Thread.

An Example.

```
#include <pthread.h>
-----
main()
{ pthread_t  thread1;
  pthread_create(&thread1, NULL, (void *) funct_1 , (void *) &r1);
  .....
  pthread_join(thread1, NULL);
-----
}
```

Note if main (or parent) thread returns before other threads complete, they will be aborted.

So do not return -- use pthread_exit(); In this case, the parent lingers until all created threads Complete.

pthread_join: The program creates a thread by calling pthread_create, and then waits for it by calling pthread_join.

Termination Of A Thread

A thread terminates normally after completing the execution of the start_routine. You can also explicitly terminate a thread using either the Pthread_exit or Pthread_Cancel

```
void pthread_exit (void *status);
```

This function terminates the thread that is making the call, and returns the value passed via the status to any waiting threads. Note that the variable containing the value to be returned should not be an automatic variable, since the value may be needed even after the thread quits. The control does not return to the program that created the thread.

```
int pthread_cancel (pthread_t thread);
```

This function cancels (or kills) a thread specified by the thread id.

The cancellation may not take effect immediately because

- holding locks

- performing clean-up

- the specified thread set state to

PTHREAD_CANCEL_DEFERRED

A thread can also disable cancellation.

Sequential vs. Parallel

Sequential Execution: The program is executed contiguously, which means, each instruction is executed one instruction after another based on Program counter except jump, return, and so on.

Parallel execution: It is a multiprocessor system where more than one CPU is involved and are tightly coupled. They share the computer bus, the clock, and sometimes memory and peripheral devices.

By increasing the number of processors, we hope to get more work done in less time. The speed-up ratio with N processors is not N ; rather, it is less than N . It is less than N because when multiple parts work together there is always some Overhead involved in keeping all parts working correctly.

Advantages are:

Increased throughput, Economy of sale, Increased reliability (fault tolerant is good.)

Parallel Programming 平行程式設計

What is Asynchronous?

Asynchronous means that things happen independently (concurrently) unless there's some enforced dependency. Traditional programming causes all events to occur in series unless the programmer takes “extraordinary measures”

What is concurrency?

Concurrency, which an English dictionary will tell you refers to things happening at the same time, is used to refer to things that appear to happen at the same time, but which may occur serially. Concurrency describes the behavior of threads or process on a uniprocessor (one processor) system.

What is parallelism?

Parallelism describes concurrent sequences that proceed simultaneously. It refers to things proceeding in the same direction independently (without intersection.)

True parallelism can occur only on a multiprocessor system, but concurrency can occur on both uniprocessor and multiprocessor system.

Parallel Programming 平行程式設計

Threads imply concurrency

So, why should we use concurrency for programming?

Multitasking has been used by Operating Systems to better utilize processor capabilities

Better Throughput and Response Time

What is the difference?

Throughput : Number of tasks/transactions completed per unit time

Response Time: The average time it takes to receive a “reply” to a request.

When one activity is waiting on some action from the kernel or from an external system, we can initiate a concurrent activity. This increases the utilization of available resources.

For example we can assign a separate task/thread to process user command inputs, while other tasks/threads can do the actual processing.

This eliminates the need for “freezing” keyboard while processing a previous request.

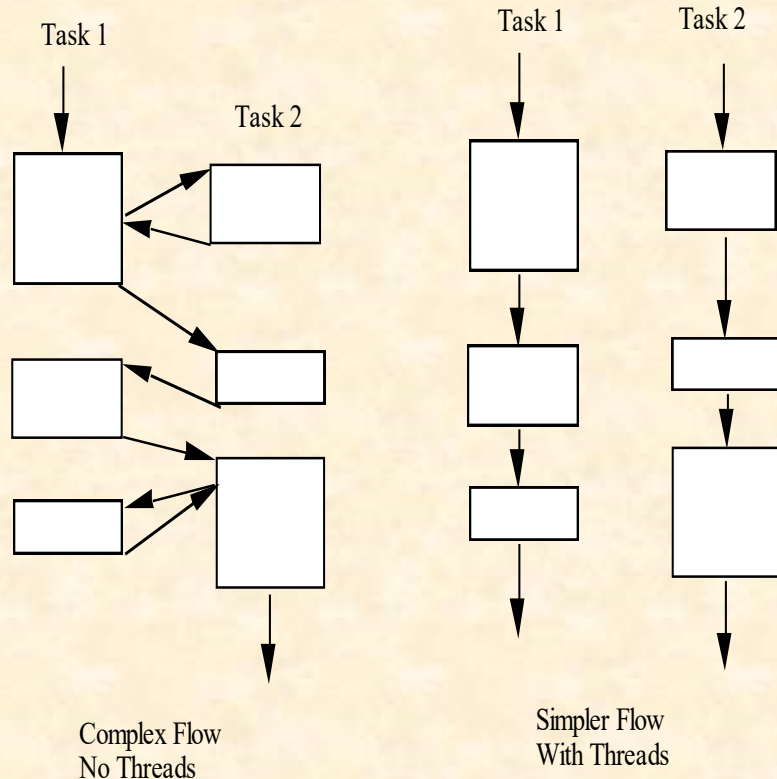
Parallel Programming

平行程式設計

Improved program structure

As can be seen, when a complex program can be viewed as concurrent activities, the flow of control becomes cleaner.

Without threads, we can still simulate the concurrent activities by interleaving the execution -- or use “Co-routines” if the language supports. Such structures cannot utilize multiple CPU's or Kernel threads; threads can use multiple processors or kernel threads



Parallel Programming 平行程式設計

- Parallelism

If multiple processors are available, we can increase the performance by using multiple concurrent (either independent or dependent) activities.

Not all applications are amenable to concurrent implementations.

Inherently concurrent systems

Independent tasks.

We can view a program development system as a concurrent system.

Debugger, GUI interface, Interactive inspector,
dynamic call grapher, performance/trace monitor

Servers.

A server needs to handle different types of requests. Can design a separate task/thread for each type of request.

Can fork new task/thread for each client

Parallel Programming 平行程式設計

Inherently concurrent systems

Simulations and Scientific Computations

Usually involve several subsystems -- each can be modeled and simulated by a separate task/thread.

A computation (e.g., Matrix Multiply) can be broken into several independent subcomputations.

Difficult concurrent systems.

Some Numerical and scientific applications are not apparently concurrent. Some modifications to the algorithms may make them concurrent.

Automatic Extraction of Concurrency

Parallel Programming 平行程式設計

When not to use Concurrency

Since concurrent programs (running on a single processor) incur overhead for context switching, it may not make sense for some programs to use concurrency.

Lot of dependencies between the concurrent activities

An activity cannot proceed until some other activity provides information

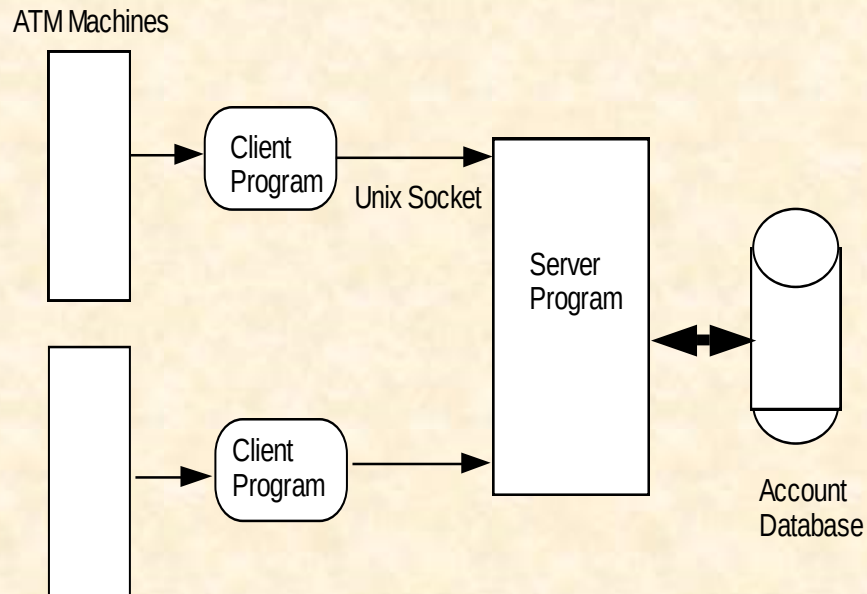
Compute Intensive.

If each activity is compute intensive it may not make much sense to use concurrency *without parallel hardware*

Parallel Programming 平行程式設計

An Example Of Concurrency (more for multithreading)

Consider a fictional Automatic Teller Machine system.



Here we have several client systems that provide an interface to the user and send user request to the server system, receive replies from the server, and relay the response to the user. The server system is responsible for checking authorization, checking for balance and updating user accounts.

Parallel Programming 平行程式設計

Automatic Teller Machine

We have identified independent activities between the multiple clients and servers.

We can also think of the server system itself as multiple tasks
one checking the user identification,
one checking the account balance,
one to communicate with the client.

We may even consider multiple concurrent tasks to access the database -- say for example all accounts for names A-K, K-R, R-Z, etc.

By doing so, we can overlap the I/O (accessing database) or communication (with Client Systems) with processing (accessing accounts and updating accounts).

We may even want to “fork” a new activity on demand.

The server system may be implemented on a SMP.

Parallel Programming 平行程式設計

Different Concurrent Programming Models

Master-Worker Model

- Main creates worker threads

- Assigns work for the worker threads

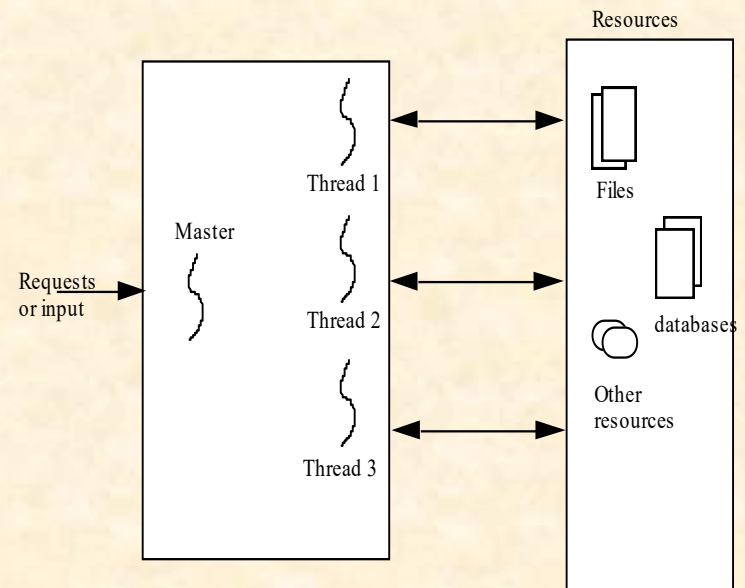
- Waits for the worker threads to complete

- Perform clean-up and finalization

- Completed worker threads are deleted

- Worker threads can be created dynamically

- Worker threads can be homogeneous or heterogeneous



Parallel Programming 平行程式設計

Consider the ATM example.

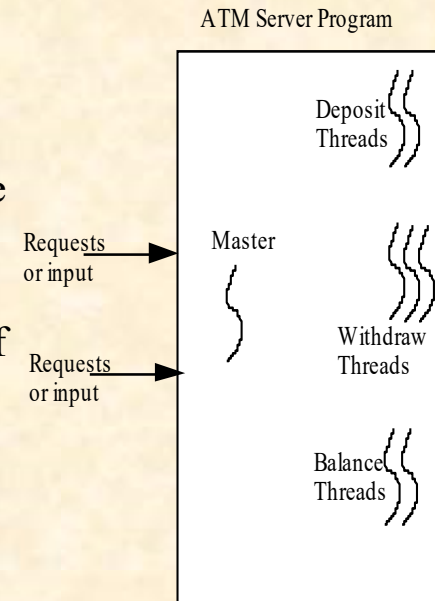
We can consider a Master thread in the ATM server.
This thread interfaces with clients, and receives requests

For each request, Master creates a Worker thread to process the request.

The worker thread can be homogeneous -- service any type of request.

Or heterogeneous. We can think of different worker threads for different types of requests.

one for withdraw requests,
one for deposit requests
one for balance enquiries.



Parallel Programming 平行程式設計

Pool Of Workers Model

A fixed number of worker threads are created at start
Even the creating/main thread becomes a worker

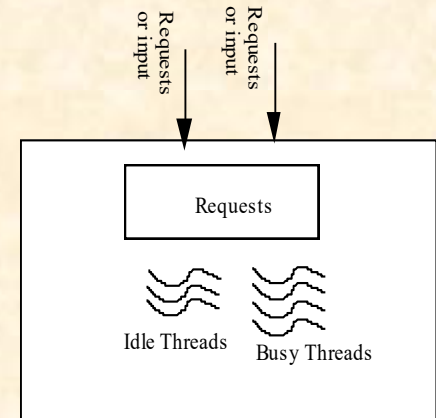
Worker thread perform pending work

When completed, a worker thread tries to find additional work or blocks for work

Worker threads are not deleted

May not have any synchronization with “parent”

Any worker can perform any work.



Consider the ATM example again.

We can consider a fixed number of worker threads within the server.

Client requests are bound or sent to different worker threads.

Worker thread can be blocked waiting for client requests.

We can use either condition variables or signals for blocking.

On completing a request, the worker thread blocks awaiting more work.

Parallel Programming 平行程式設計

Pipeline Model

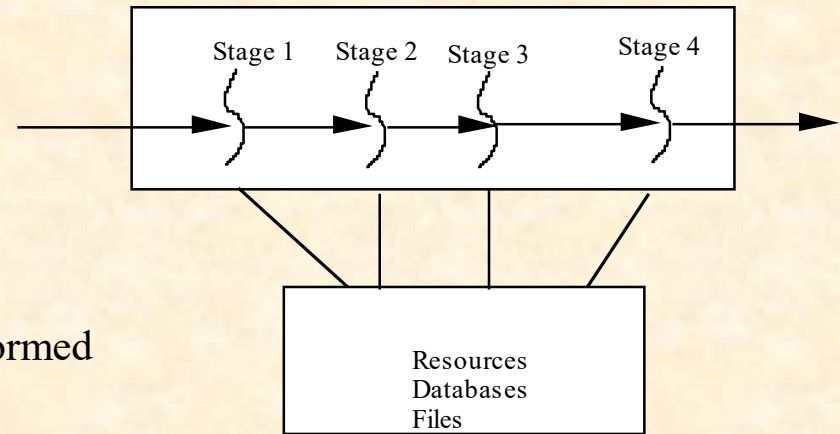
Here we have a fixed number of threads
Each thread performs a different task
A thread completes its work on current data
feeds its results to the next thread

Useful when the **same sets of computations** are performed repeatedly on different data sets

Can we think of pipelined implementation of the ATM example?

Consider the various function in processing user requests.

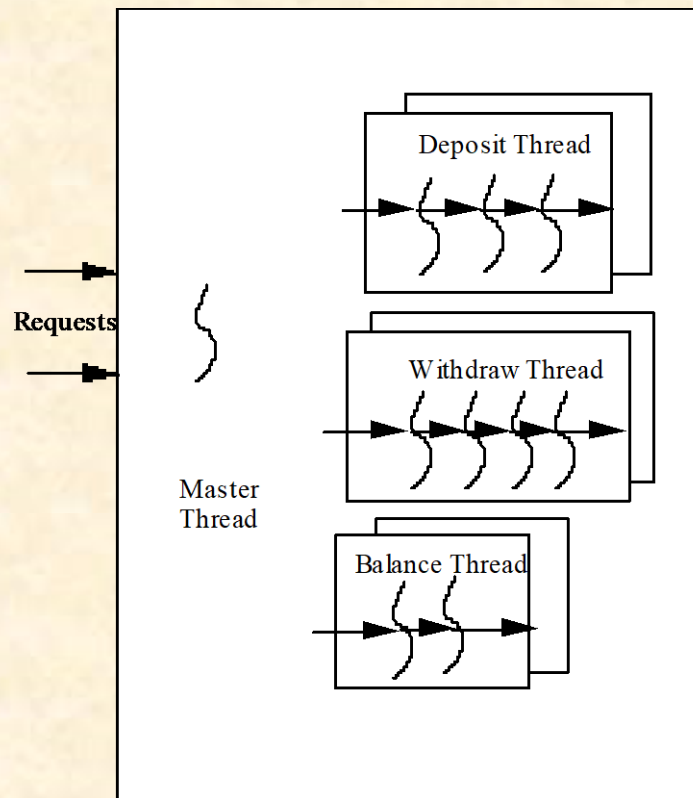
- Communication with client,
- Authentication
- Search database for account information
- Process request: withdraw, deposit or balance request.
- Process errors
- Communicate results with clients.



Parallel Programming 平行程式設計

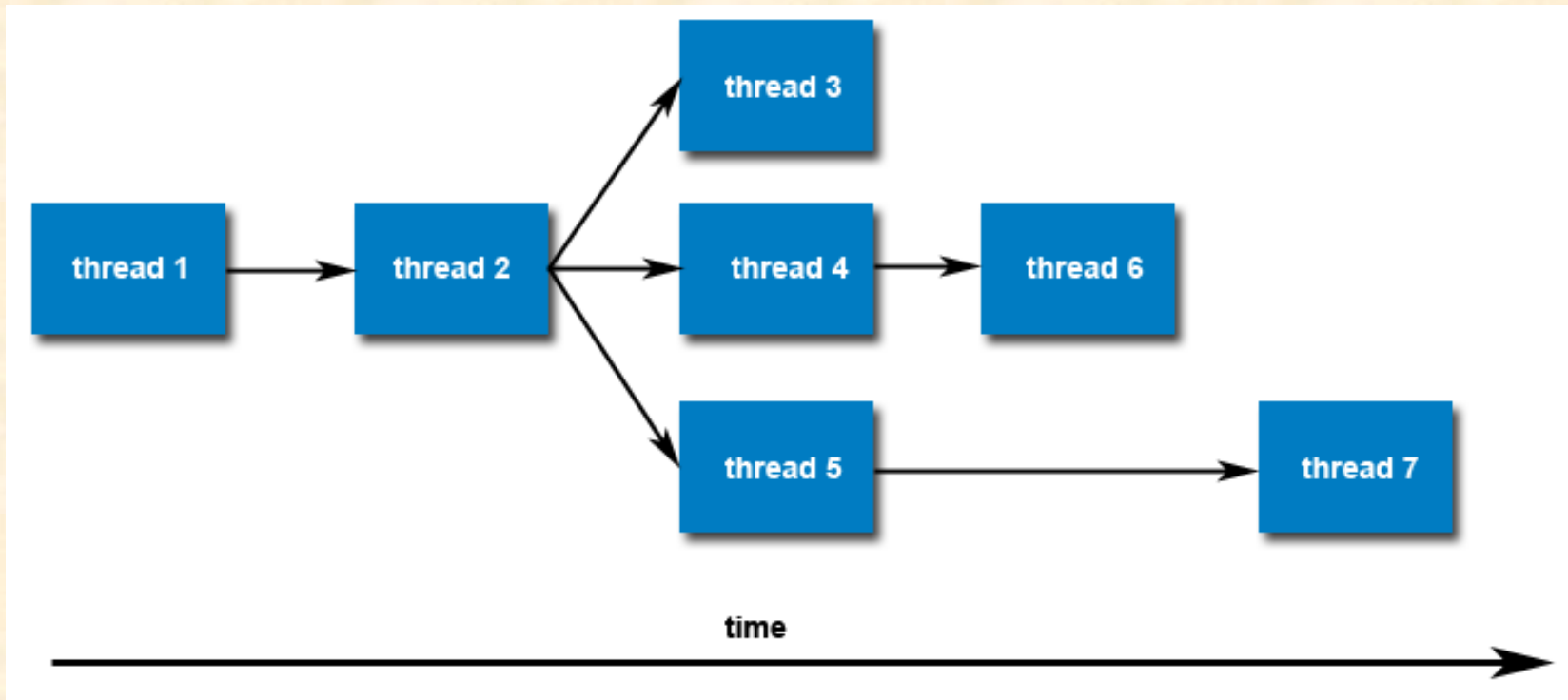
Hybrid models.

We can combine the pipelined model with pool of workers or master-slave models



Multithreading Model

Once created, threads are peers, and may create other threads. There is no implied hierarchy or dependency between threads.



Multithreading and parallelism at different levels

Language level

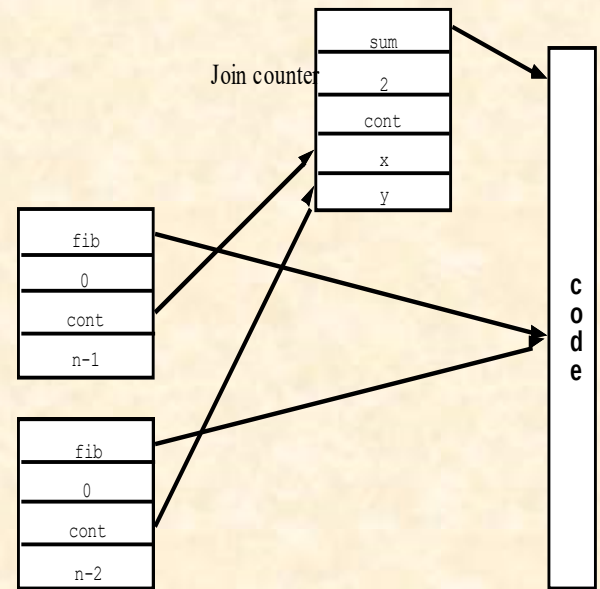
Java, Ada-95, Cilk, HPF, MP-Fortran, MP-C

Language constructs to create, spawn, synchronize

An Example

```
thread fib (cont int k, int n)
{
    if (n < 2)
        send_argument (k, n)
    else {
        cont int x, y;
        spawn_next sum (k, ?x, ?y); /* create a successor thread
        spawn fib (x, n-1);          /* fork a child thread
        spawn fib (y, n-2);          /* fork a child thread
    }

thread sum (cont int k, int x, int y)
{
    send_argument (k, x+y); /* return results to parent's successor
}
```



Let us start programming in Multithreading

Write a simple Multithread Program to print Hello.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#define NUM_THREADS 8
void *PrintHello(void *threadid)
{
    long tid;
    tid = (long)threadid;
    printf("Hello World! It's me, thread #%ld!\n", tid);
    pthread_exit(NULL);
}
int main (int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    for(t=0; t<NUM_THREADS; t++){
        printf("In main: creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc){
            printf("ERROR; return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
    }
    /* Last thing that main() should do */
    pthread_exit(NULL);
}
```

Take a Simple example program and see how we can write Using multithread

Following is a simple program that adds from 0 to 99

We can write in a simple c language as follows;

```
#include <stdio.h>
main()
{
    int i, sum=0;

    for(i=0; i<100; i++)
        sum = sum + i ;

    printf("The total is %d\n", sum);
}
```

Multithreading and parallelism at different levels

Runtime Libraries

Pthreads, DCE threads, Solaris threads, Linux threads

Insert library calls to create, spawn synchronize

An Example.

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
```

```
void simple_add(int *input);
```

```
int x=0;
```

```
int main(void)
```

```
{
```

```
    pthread_t thread1;
```

```
    if(pthread_create(&thread1,NULL,(void *)simple_add,(void *)&x)!=0)
        perror("pthread_create"),exit(0);
```

```
    pthread_join(thread1,NULL);
```

```
    printf("x = %d \n",x);
```

```
/* Last thing that main() should do */
```

```
pthread_exit(NULL);
```

```
    return 0;
```

```
}
```

```
void simple_add(int *input)
```

```
{
```

```
    int i,y=0;
```

```
    for(i=0;i<100;i++)
```

```
        y=y+i;
```

```
    *input=y;
```

```
    // (you can also return *input=y;)
```

```
}
```

What platform can we use this program to compile? How to compile this program?

If you have no Linux machine, you can ask for an account in a Linux machine. I can give you an account on the Linux machine. Try these programs quick before we can try some other real parallel program on MPI and PVM.

Warning! Do not wait too long and you can't catch up with many program examples and it will be too late.

Instruction to write a multithreaded program in a Linux machine.

Linux machine uses POSIX standard.

Type the above mentioned programs that I gave in class.

To compile for the pthread program use the `#include <pthread.h>` thread library needed for the pthread program.

Compiling a program would be as follows:

If your program is called `pthreadone.c`

`gcc pthreadone.c -lpthread` creates an `a.out` executable file.

Or `gcc -o pthreadone pthreadone.c -lpthread` creates a `pthreadone`

Run the executable files as you run a c program and see the output if it is correct!

Pthread Creation and Termination Example

Hello world

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
void *PrintHello(void *threadid)
{
    long tid;
    tid = (long)threadid;
    printf("Hello World! It's me, thread #%ld!\n", tid);
    pthread_exit(NULL);
}
int main (int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int rc; long t;
    for(t=0; t<NUM_THREADS; t++)
    {
        printf("In main: creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc)
        {
            printf("ERROR; return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
    } /* Last thing that main() should do */
    pthread_exit(NULL);
}
```

Part1: What is the difference between Single Process (program), Multiprocessing and Multithreading?

To illustrate the difference we will consider an example.

Let us take a simple program which shows two functions in a program.

Then we rewrite the program using processes.

The same program can be written using multithread.

```

/*Creates N > 2 processes. */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
Int BUF_SIZE =20;
int main(int argc, const char * argv[])
{ pid_t pid; int N_PROC =2;          /* * Work of a single process. */
  for(int i = 0; i < N_PROC; i++) void ProcessWork()
  {
    pid = fork();
    if(pid == 0)
    {
      ProcessWork();
      exit(0);
    }
  }
}

void ProcessWork()
{
  char buffer[BUF_SIZE];
  for (int i = 1; i <= 5; i++)
  {
    sleep(1);
    sprintf(buffer, "PID = %d, count = %d\n", getpid(), i);
    write(1, buffer, strlen(buffer));
  }
}

```

```

#include <stdio.h>
#include <stdlib.h>
void first_function(int *);
void second_function(int *);
void final_function(int , int);
int i =0, j =0;

main()
{
first_function(&i);
second_function(&j);
final_function(i,j);
}

void first_function(int *num)
{
int x,y,z;
for(x=0; x< 10; x++)
{
printf("Inside the first_function\n");
for(y=0; y< 1000; y++)
z =z+1; /* introduce delay*/
(*num)++;
}
}

```

```

void second_function(int *num)
{
int x,y,z;
for(x=0; x< 10; x++)
{
printf("Inside the second_function\n");
for(y=0; y< 1000; y++)
z =z+1; /* introduce delay*/
(*num)++;
}
}

void final_function(int first, int second)
{
int final_score;
final_score = first + second ;
printf("In final : first = %d, second = %d and final_score = %d\n", first, second, final_score);
}

```

Three functions doing the task.

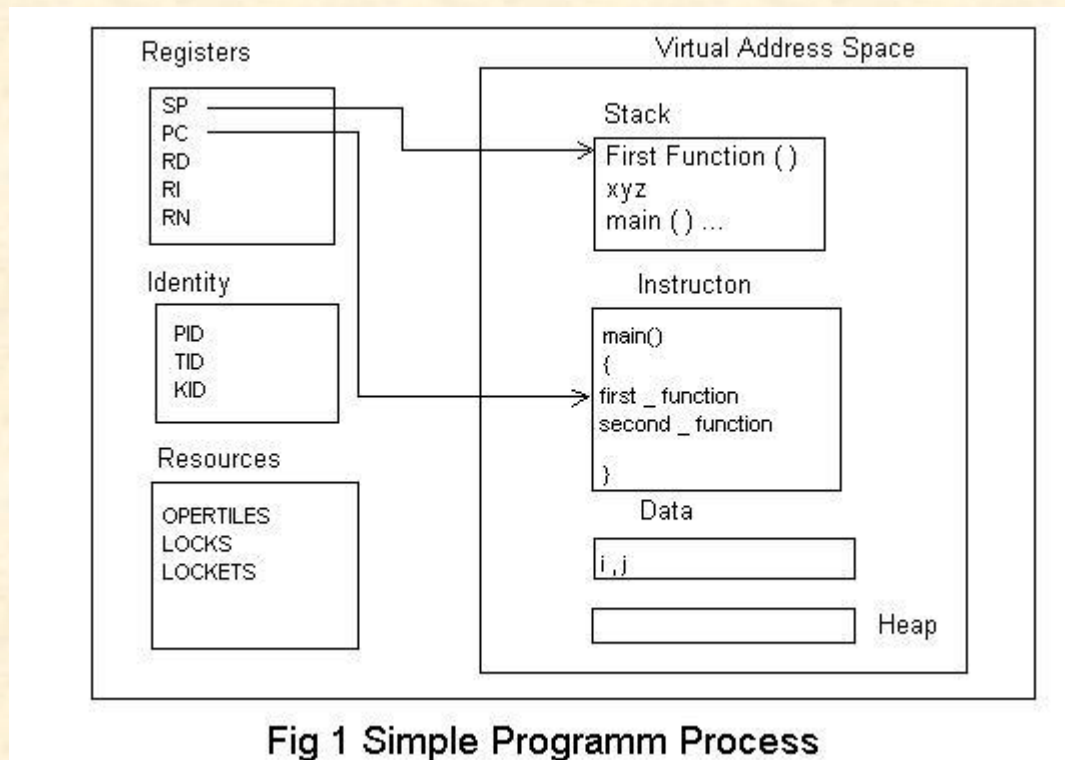
**The figure below shows the layout of this program in the virtual address memory.
The various memory areas are :**

A read-only memory for the machine language instructions of the program

A read-write area for the global variables

A heap area for memory that is dynamically allocated (example *malloc* calls)

A stack on which local variables of current functions are maintained



Unix Multiprocessing:

We can rebuild the first program to illustrate the unix multiprocessing.

The *main* routine starts in a single process which is the *parent* process.

The parent process then creates the child process to execute the *first_function* and another child process to execute the *second_function* routines.

Then the parent process waits for both of its child process to finish and then calls *final function*


```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
void first_function(int *);
void second_function(int *);
void final_function(int,int);
int shared_mem_id;
int *shared_mem_ptr;
int *i, *j;
main()
{
pid_t child1_pid, child2_pid;
int state; /* initialise the shared memory segment */
shared_mem_id = shmget(IPC_PRIVATE, 2*sizeof(int),0);
shared_mem_ptr = (int *)shmat(shared_mem_id, (void*)0, 0);
/* giving the pointers to the variables */
i = shared_mem_ptr;
j = (shared_mem_ptr + 1);
*i = 0; *j =0;

```

```

/* start the first child process */
if((child1_pid = fork())== 0)
{ /* job of first child*/
first_function();
exit();
}

```

```

/*start the second child process*/
if((child2_pid = fork())== 0)
{ /* job of second child*/
second_function();
exit();
}
/*parent*/
waitpid(child1_pid, state,0);
waitpid(child2_pid, state,0);
final_function(*i, *j);
}

```

```
void first_function(int *num)
{
    int x,y,z;
    for(x=0; x< 10; x++)
    {
        printf("Inside the first_function\n");
        for(y=0; y< 1000; y++) z =z+1;
        /* introduce delay*/ (*num)++;
    }
}

void second_function(int *num)
{
    int x,y,z; for(x=0; x< 10; x++)
    {
        printf("Inside the second_function\n");
        for(y=0; y< 1000; y++) z =z+1;
        /* introduce delay*/ (*num)++;
    }
}

void final_function(int first, int second)
{
    int final_score;
    final_score = first + second ;
    printf("In final : first = %d, second = %d and final_score =
%d\n", first, second, final_score);
}
```

the UNIX system calls that create a new process is called *fork*.

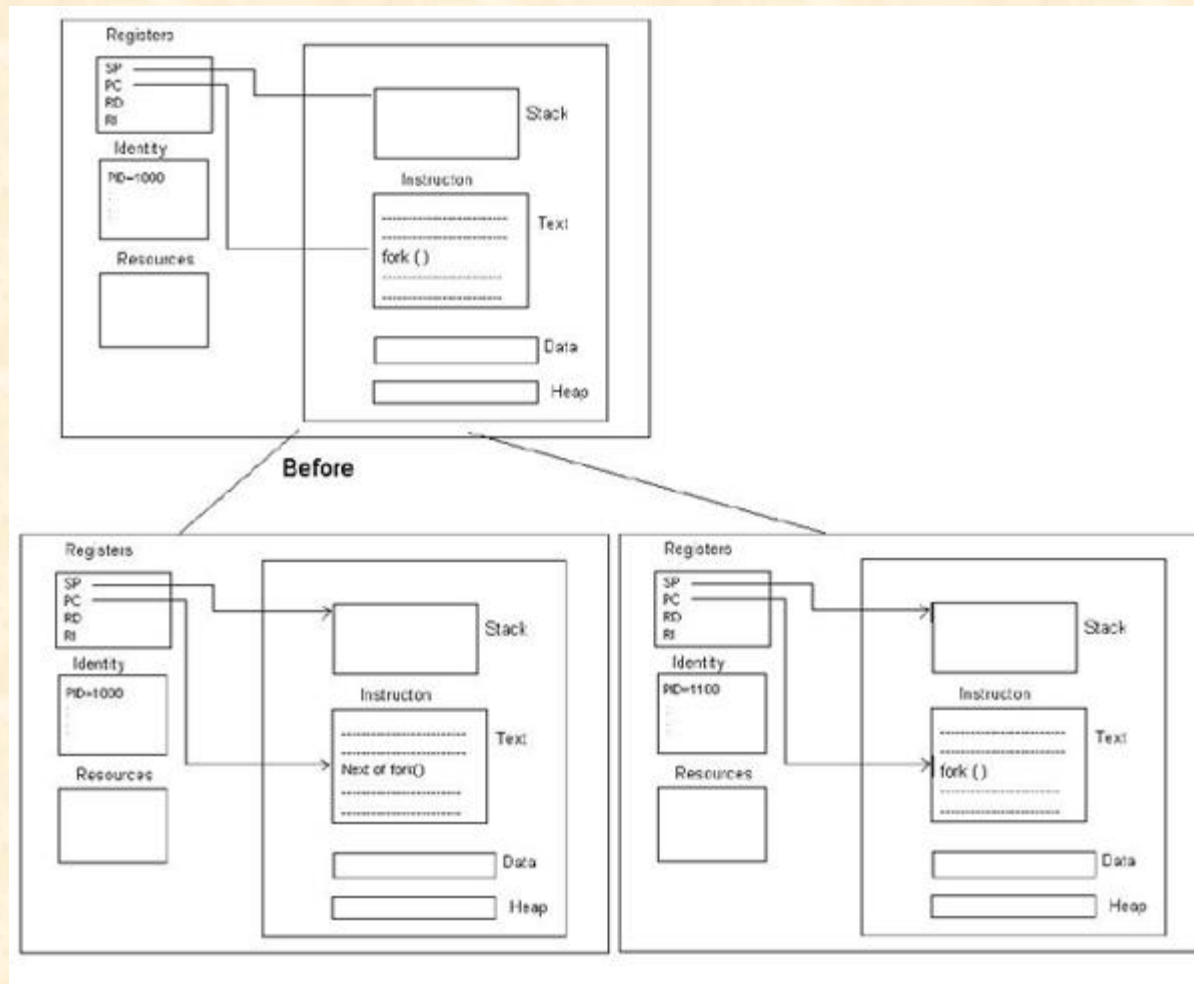
the *fork* creates the child process that is identical to its parent process at the time of function call of *fork* . following are the result

The child has its own process identifier, or PID

the fork call returns different values to child and parent processes

(returns a value of 0 to child and child's PID to parent)

As depicted in the figure, as soon as the program forks into two different processes, the parent and child process execute independently unless there is synchronization.



Pthreads Concurrent Programming:

Pthreads is a standardized model for dividing a program into subtasks which can be run in parallel. We will repeat the example using Pthreads. The program starts with the main thread. Using the Pthread function calls, the creator thread spawns a thread to execute the `first_function` and another to execute the `second_function` routine.

```
#include <pthread.h>
```

```
void first_function(int *);  
void second_function(int *);  
void final_function(int,int);
```

```
int i=0; j=0;
```

```
main()
```

```
{  
    pthread_t thread1, thread2;  
    pthread_create(&thread1, NULL, (void*) first_function, (void*)&i);  
    pthread_create(&thread2, NULL, (void*) second_function, (void*)&j);  
  
    pthread_join(thread1, NULL);  
    pthread_join(thread2, NULL);  
  
    final_function(i,j);  
}
```



```
void first_function(int *num)
```

```
{
```

```
    int x,y,z;
```

```
    for(x=0; x< 10; x++)
```

```
    {
```

```
        printf("Inside the first_function\n");
```

```
        for(y=0; y< 1000; y++)
```

```
        z =z+1; /* introduce delay*/
```

```
        (*num)++;
```

```
    }
```

```
}
```

```
void second_function(int *num)
```

```
{
```

```
    int x,y,z;
```

```
    for(x=0; x< 10; x++)
```

```
    {
```

```
        printf("Inside the second_function\n");
```

```
        for(y=0; y< 1000; y++)
```

```
        z =z+1; /* introduce delay*/
```

```
        (*num)++;
```

```
    }
```

```
}
```

```
void final_function(int first, int second)
```

```
{
```

```
    int final_score;
```

```
    final_score = first + second ;
```

```
    printf("In final : first = %d, second = %d  
    and final_score = %d\n",
```

```
    first, second, final_score);
```

```
}
```

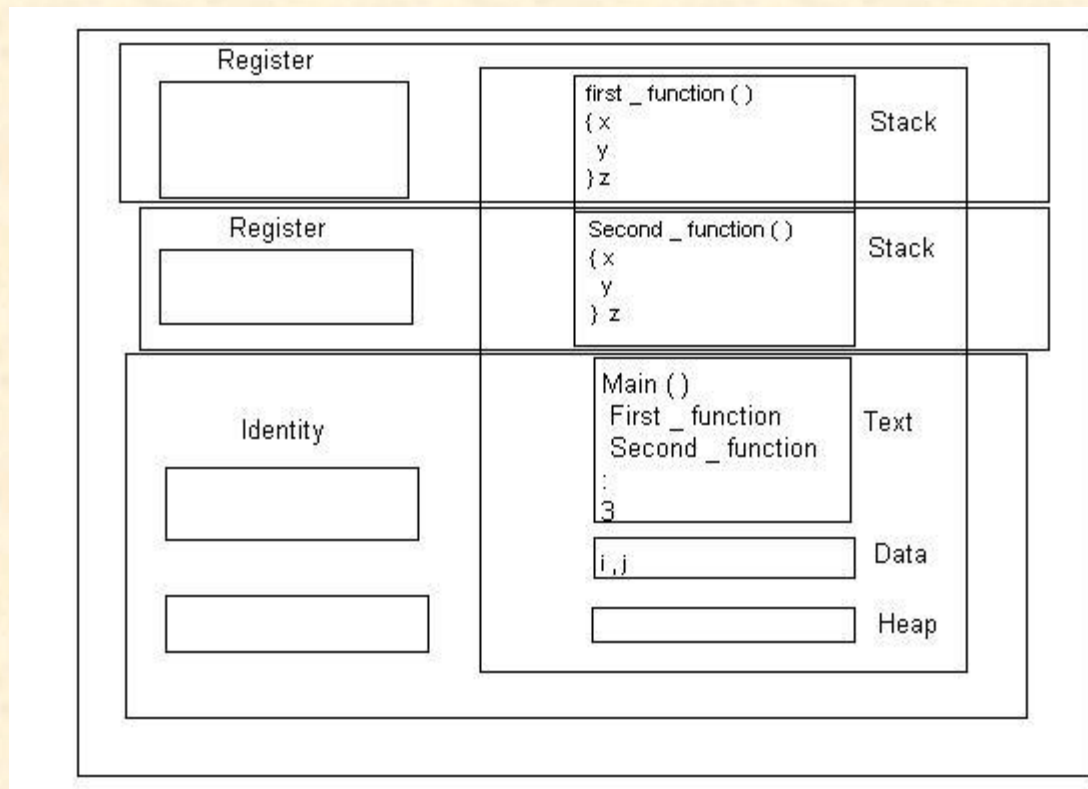
The pthread_create uses the following parameters

A pointer to a buffer which identifies the newly created thread(Type of pthread_t*).

A pointer to a structure of type thread attribute object.

A pointer to routine at which the new thread will start executing

A pointer to a parameter to be passed to the routine at which the new thread starts
threads are created without replicating an entire process



Thread Synchronization

The threads library provides three synchronization mechanisms:

- **mutexes** - Mutual exclusion lock:
 - Block access to variables by other threads. This enforces exclusive access by a thread to a variable or set of variables.
- **joins** - Make a thread wait till others are complete (terminated).
- **condition variables** - data type `pthread_cond_t`

Refer to the following link for more understanding and writing posix thread programs.

<http://www.yolinux.com/TUTORIALS/LinuxTutorialPosixThreads.html#BASICS>

Example of mutexes

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
void *functionC();
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;
main()
{
    int rc1, rc2;
    pthread_t thread1, thread2;

    /* Create independent threads each of which will execute functionC */
    if( (rc1=pthread_create( &thread1, NULL, &functionC, NULL)) )
    {
        printf("Thread creation failed: %d\n", rc1);
    }
    if( (rc2=pthread_create( &thread2, NULL, &functionC, NULL)) )
    {
        printf("Thread creation failed: %d\n", rc2);
    }

    /* Wait till threads are complete before main continues. Unless we */
    /* wait we run the risk of executing an exit which will terminate */
    /* the process and all threads before the threads have completed. */
    pthread_join( thread1, NULL);
    pthread_join( thread2, NULL);
    exit(0);
}
```

Example of mutexes (cont..)

```
void *functionC()
{
    /*When we increment the counter, we need to allow */
    /* only one thread to increment the counter, hence */
    /* we need to lock the global variable */
    pthread_mutex_lock( &mutex1 );
    counter++;
    printf("Counter value: %d\n",counter);
    pthread_mutex_unlock( &mutex1 );
}
```

What are Mutexes?

Mutexes are used to prevent data inconsistencies due to race conditions. A race condition often occurs when two or more threads need to perform operations on the same memory area, but the results of computations depends on the order in which these operations are performed. Mutexes are used for serializing shared resources. Anytime a global resource is accessed by more than one thread the resource should have a Mutex associated with it. One can apply a mutex to protect a segment of memory ("critical region") from other threads. Mutexes can be applied only to threads in a single process and do not work between processes as do semaphores

MUTEXES

When a mutex lock is attempted against a mutex which is held by another thread, the thread is blocked until the mutex is unlocked. When a thread terminates, the mutex does not unless explicitly unlocked. Nothing happens by default.

Joins

A join is performed when one wants to wait for a thread to finish.

A thread calling routine may launch multiple threads then wait for them to finish to get the results. One wait for the completion of the threads with a join.

Example of Joins

```
#include <stdio.h>
#include <pthread.h>
#define NTHREADS 10
void *thread_function(void *);
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

main()
{
    pthread_t thread_id[NTHREADS];
    int i, j;
    for(i=0; i < NTHREADS; i++)
    {
        pthread_create( &thread_id[i], NULL, thread_function, NULL );
    }
    for(j=0; j < NTHREADS; j++)
    {
        pthread_join( thread_id[j], NULL);
    }

    /* Now that all threads are complete I can print the final result. */
    /* Without the join I could be printing a value before all the threads */
    /* have been completed. */
    printf("Final counter value: %d\n", counter);
}
```

```
void *thread_function(void *dummyPtr)
{
    printf("Thread number %ld\n", pthread_self());

    pthread_mutex_lock( &mutex1 );
    counter++;
    pthread_mutex_unlock( &mutex1 );
}
```

Condition Variables

A condition variable is a variable of type `pthread_cond_t` and is used with the appropriate functions for waiting and later, process continuation. The condition variable mechanism allows threads to suspend execution and relinquish the processor until some condition is true. A **condition variable** must always be associated with a **mutex** to avoid a race condition created by one thread preparing to wait and another thread which may signal the condition before the first thread actually waits on it resulting in a deadlock. The thread will be perpetually waiting for a signal that is never sent. Any mutex can be used, there is no explicit link between the mutex and the condition variable.

Functions used in conjunction with the condition variable

Creating/Destroying:

pthread_cond_init

pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

pthread_cond_destroy

Waiting on condition:

pthread_cond_wait

pthread_cond_timedwait - place limit on how long it will block.

Waking thread based on condition:

pthread_cond_signal

pthread_cond_broadcast - wake up all threads blocked by the specified condition variable.

Example using condition variable

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
pthread_mutex_t count_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t condition_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t condition_cond = PTHREAD_COND_INITIALIZER;
```

```
void *functionCount1();
void *functionCount2();
```

```
int count = 0;
#define COUNT_DONE 10
#define COUNT_HALT1 3
#define COUNT_HALT2 6
```

```
main()
{
    pthread_t thread1, thread2;
    pthread_create( &thread1, NULL, &functionCount1, NULL);
    pthread_create( &thread2, NULL, &functionCount2, NULL);

    pthread_join( thread1, NULL);
    pthread_join( thread2, NULL);
    exit(0);
}
```


Functioncount1()

```
void *functionCount1()
{
    for(;;)
    {
        pthread_mutex_lock( &condition_mutex );
        while( count >= COUNT_HALT1 && count <= COUNT_HALT2 )
        {
            pthread_cond_wait( &condition_cond, &condition_mutex );
        }

        pthread_mutex_unlock( &condition_mutex );

        pthread_mutex_lock( &count_mutex );
        count++;
        printf("Counter value functionCount1: %d\n",count);
        pthread_mutex_unlock( &count_mutex );

        if(count >= COUNT_DONE) return(NULL);
    }
}
```

Functioncount2()

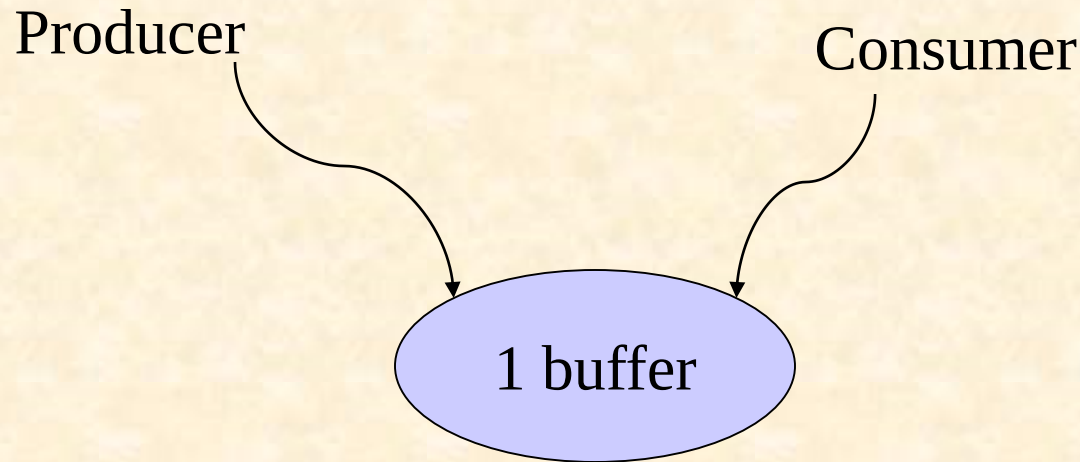
```
void *functionCount2()
{
    for(;;)
    {
        pthread_mutex_lock( &condition_mutex );
        if( count < COUNT_HALT1 || count > COUNT_HALT2 )
        {
            pthread_cond_signal( &condition_cond );
        }
        pthread_mutex_unlock( &condition_mutex );

        pthread_mutex_lock( &count_mutex );
        count++;
        printf("Counter value functionCount2: %d\n",count);
        pthread_mutex_unlock( &count_mutex );

        if(count >= COUNT_DONE) return(NULL);
    }
}
```

Let us see how we can use multithreads in a meaningful way that a program can use them and accomplish two different tasks.

Producer and consumer problem that we studied in the operating system Multiple processes.



What should be considered here in the above program? The buffer is Shared by producer and the consumer.

- 1. When the producer produces, consumer should not consume or consumer consumes producer should not produce on the buffer. Use a **lock** to protect buffer.**
- 2. The producer places something in the buffer, then it must **signal** the consumer that there is something in the buffer and it can consumer.**

Note: Since we have one buffer, we only need **one lock** variable and **one signal** variable.

Using Mutual Exclusion locks

mutex

Algorithm

```
Void Producer () {  
  Int Buf;  
  
  Loop {  
    pthread_lock (&Mutex_B);  
  
    Check_Free_Buf;  
    If No_Buffer Then  
      pthread_UnLock (&Mutex_B);  
  
    Else {  
      Read_Input_Data (Buf);  
      Create_Full_Buf(Buf);  
      pthread_UnLock (&Mutex_B);  
    }  
    Exit When No_more_data;  
  }  
}
```

```
Void Consumer {  
  Int F_Buf;  
  
  Loop {  
    pthread_lock (&Mutex_B);  
  
    Check_Full_Buf;  
    If No_Full_Buf Then  
      pthread_UnLock(&Mutex_B);  
  
    Else{  
      Print_Data(Buf);  
      Release_Empty_Buf(Buf);  
      pthread_UnLock(&Mutex_B);  
    }  
    Exit When ?  
  }  
}
```

Other way of synchronizing access to Data.

Message Passing

Procedure XYZ Is

task body Producer is

Buf: Character;

**Begin
Loop**

_Receive Free_Buf;
 Read_Input_Data (Buf);
 Send Full_Buf;
 Exit When No_more_data;

**End Loop;
End Producer;**

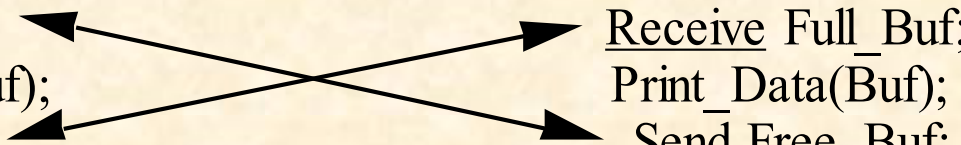
task body Consumer is

F_Buf : Character;

**Begin
Loop**

Receive Full_Buf;
 Print_Data(F_Buf);
 _Send Free_Buf;
 Exit When ?

**End Loop;
End Consumer;**



Problem of Producer and consumer : Written in multithread

page:1

First program about producer consumer where the producer produces and the consumer consumes. one random number is produced by the producer and the number is consumed by the consumer.

AUTHOR: JOE ARUL

PROGRAM: PTHREAD PROGRAM

*****/

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
/* this is the data structure that is used between the producer and  
consumer threads */
```

```
typedef struct BUF {
```

```
    int numb;
```

```
    int total;
```

```
    int occ;
```

```
    } data;
```

```
data Buf={0,0,0};
```

```
pthread_mutex_t buflock=PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_cond_t sigdata=PTHREAD_COND_INITIALIZER;
```

```
/* function prototype */
```

```
void producer(void);
```

```
void consumer(void);
```


Problem of Producer and consumer : Written in multithread

page:2

```
main(int argc, char **argv)
{
    int numb;
    pthread_t cons_thr; pthread_t prod_thr;

    pthread_setconcurrency(2);

    /* create the producer and consumer threads */
    pthread_create(&prod_thr, NULL, (void *)producer, NULL);
    pthread_create(&cons_thr, NULL, (void *)consumer, NULL);

    /* wait for the producer and consumer to finish */
    pthread_join(prod_thr, NULL);
    pthread_join(cons_thr, NULL);

    /* exit the program */
    return(0);
}
```

page:3

```
void producer(void )
{
    int rand(void);
    int numb,i,flag=0;
    /* Producer is made to produce 100 random numbers and quit the program*/
    while(!flag)
    {
        /* the producer ! lock the mutex */
        pthread_mutex_lock(&buflock);

        /* check to see if any buffers are empty */
        /* If not then wait for that condition to become true */
        while (Buf.occ == 1)
            pthread_cond_wait(&sigdata, &buflock);
        /* read from the file and put data into a buffer */

        numb= rand();
        if(Buf.total==100) {
            flag=1;
            Buf.numb=-1;
        } else {
            Buf.numb=numb;
            printf("Produced Number %d\n",Buf.numb);
        }
        /* Just produce given number of random numbers */

        Buf.total++;

        /* increment the number of buffers that are filled */
        /* Here we just have one number to produce and consume */
        Buf.occ++;

        /* signal the consumer to start consuming */
        pthread_cond_signal(&sigdata);
        /* release the buffer mutex */
        pthread_mutex_unlock(&buflock);
        /* leave the while loop */
    }
}
```

Page 4: **/* The consumer thread */**

void consumer(void)

{

int flag=0;

/* check to see if any buffers are filled or if the done flag is set */

/* In this example: only one number is generated and the consumer consumes*/

while (!flag) {

/* lock the mutex */

pthread_mutex_lock(&buflock);

/* check to see if any buffers are filled */

/* if not then wait for the condition to become true */

while(Buf.occ == 0)

pthread_cond_wait(&sigdata, &buflock);

/* write the data from the buffer to the file */

if(Buf.numb == -1)

flag=1;

else

printf("Consumed Number %d\n",Buf.numb);

/* decrement the number of buffers that are full */

Buf.occ=0;

/* signal the producer that a buffer is empty */

pthread_cond_signal(&sigdata);

/* release the mutex */

pthread_mutex_unlock(&buflock);

}

}

What is this condition variable that we are using in the program?

Condition Variables and Events in threads

Unlike Mutex variable, which can be used to protect a critical section, Condition Variable can be used to check or wait for a specific condition or event to happen.

For example, a thread can sleep until some condition is met -- say, when the value of a variable exceeds some predefined value.

For example, when there are no new items produced by the producer, the consumer thread can sleep until a new item is produced.

We can view this somewhat similar to Unix signals. The main difference being, a thread waiting for a condition is expressing a willingness to handle the condition, while a Unix signal interrupts the process and forces handling of the signal.

You must declare condition variables using `pthread_cond_t`
and (as with mutex) initialize them

```
pthread_cond_t my_condition_var = PTHREAD_COND_INITIALIZER;
```

Usage of condition variable together with the mutex

Condition Variables and Events

```
pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t my_condition_var = PTHREAD_COND_INITIALIZER;
;
int static my_condition;
```

```
...
/* Thread_1 */
```

```
pthread_mutex_lock (&my_mutex);
```

```
while (!my_condition)
```

```
pthread_cond_wait
```

```
    (&my_condition_var,
     &my_mutex);
```

```
....
```

```
pthread_mutex_unlock (&my_mutex);
```

```
--
/* Thread_2 */
```

```
----
```

```
pthread_mutex_lock (&my_mutex);
```

```
If (...)
```

```
    my_conditon = TRUE;
```

```
pthread_cond_signal
```

```
    (&my_condition_var);
```

```
pthread_mutex_unlock(&my_mutex);
```

```
----
```

Note the use of Mutex variable along with condition variable.

Condition Variables and Events

Notice that when you are waiting on a condition variable, you should give up the mutex lock -- that is why you indicate which locks you are holding, if any, in the `pthread_cond_wait` call.

Producer-Consumer program using condition variables

```
Void Producer () {
  Int Buf;

  Loop {
    pthread_lock (&Mutex_B);
    Check_Free_Buf;
    If No_Buffer Then
      pthread_cond_wait
        (&producer_condition_var,
         &my_mutex);
    Else {
      Read_Input_Data (Buf);
      Create_Full_Buf(Buf);
      pthread_cond_signal
        (&consumer_condition_var);
      pthread_UnLock (&Mutex_B); }/**/
    Exit When No_more_data; }
}
```

```
Void Consumer {
  Int F_Buf;

  Loop {
    pthread_lock (&Mutex_B);
    Check_Full_Buf;
    If No_Full_Buf Then
      pthread_cond_wait
        (&consumer_condition_var,
         &my_mutex);
    Else{
      Print_Data(Buf);
      Release_Empty_Buf(Buf);
      pthread_cond_signal
        (&consumer_condition_var);
      pthread_UnLock (&Mutex_B);}
    Exit When ? }
}
```


Parallel Programming

平行程式設計

Condition Variables and Events

Condition variables are the source of most programming errors.

1. Forgetting to unlock mutex

```
Void Producer () {
  Int Buf;

  Loop {
    pthread_lock (&Mutex_B);
    Check_Free_Buf;
    If No_Buffer Then
      pthread_cond_wait
        (&my_condition_var,
         &my_mutex);
    Else {
      Read_Input_Data (Buf);
      Create_Full_Buf(Buf);
      pthread_cond_signal
        (&consumer_condition_var);}
    pthread_UnLock (&Mutex_B); /******/
    Exit When No_more_data; }
}
```

2. If Thread-2 signals before Thread-1 executes pthread_cond_wait, the signal is lost (and Thread-1 is deadlocked).

3. If multiple threads are waiting on the same signal, only one thread is released when the condition is signalled.

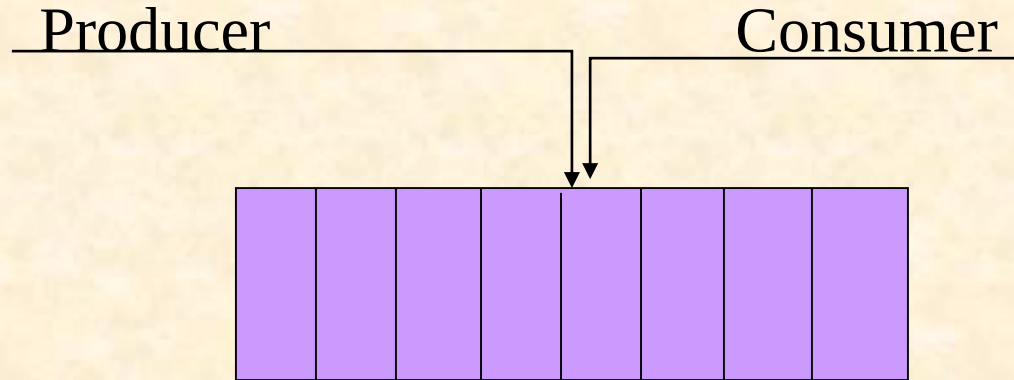
4. What happens if the “signalling” thread dies?

We can use a timed wait to specify a limited time after which the “waiting” will be released.

Producer and consumer problem 2:

Now we will have little variation of this problem

We will have 8 buffers and see what we need to do to write this program using multithread.



What should we consider in the above program? Since there are 8 buffers.

1. The Producer must check if all the 8 buffers are full, then don't produce anymore. At the same time after producing it must signal the consumer that it has produced. There are **two signals**. One is called **producer signal** and the other is called **consumer signal**. The producer uses the consumer signal. The consumer uses the producer signal.
2. The consumer must check to see, if all the buffers are empty then, don't consume. Consumer signals the producer signal to inform the producer there is one buffer empty that it can produce.

Page 1:

PRODUCER CONSUMER PROGRAM - PRODUCER produces upto 8 number and the consumer consumes if the buffer is full or else waits for the producer to produce.

PROGRAM: PTHREAD PROGRAM

*****/

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
/* this is the data structure that is used between the producer  
and consumer threads */
```

```
typedef struct BUF {
```

```
    int numb[9];  
    pthread_mutex_t buflock;  
    pthread_cond_t sigcon;  
    pthread_cond_t sigpro;  
    int done;  
    int total;  
    int tot;  
    int occ;
```

```
    } data;
```

```
/* Initialize */
```

```
data Buf={{0,0,0,0,0,0,0,0,0},
```

```
PTHREAD_MUTEX_INITIALIZER,PTHREAD_COND_INITIALIZER,PTHREAD_COND_INITIALIZER,0,0,0,0};
```

```
/* function prototype */
```

```
void producer(void);
```

```
void consumer(void);
```

Page 2:

```
main(int argc, char **argv)
{
    int numb;
    pthread_t cons_thr;
    pthread_t prod_thr;

    /* set the thread concurrency to 2 so the producer and consumer can
       run concurrently */
    pthread_setconcurrency(2);
    /* create the producer and consumer threads */
    pthread_create(&prod_thr, NULL, (void *)producer, NULL);
    pthread_create(&cons_thr, NULL, (void *)consumer, NULL);

    /* wait for the producer and consumer to finish */
    pthread_join(prod_thr, NULL);
    pthread_join(cons_thr, NULL);

    /* exit the program */
    return(0);
}
```

Page 3:

```
void producer(void )
{
    int rand(void);  int numb,flag=0;
    while(!flag)
    {
        pthread_mutex_lock(&Buf.buflock);

        /* the producer ! lock the mutex */

        /* check to see if any buffers are empty */
        /* If not then wait for that condition to become true */

        while (Buf.occ >=8)
            pthread_cond_wait(&Buf.sigpro, &Buf.buflock);

        /* read from the file and put data into a buffer */

        numb=rand();
        if(Buf.total==100)
        {
            printf("Producer done!\n");
            flag=1;
            Buf.done=1;
        } else {

            /* increment the number of buffers that are filled */
            /* Here we just have 8 numbers to produce and consume */

            Buf.occ++;
            Buf.numb[Buf.occ]=numb;
            printf("Producer produced %d Number %d\n",Buf.total,Buf.numb[Buf.occ]);
        }

        /* Just produce given number of random numbers */

        Buf.total++;

        /* release the buffer mutex */

        pthread_mutex_unlock(&Buf.buflock);

        /* signal the consumer to start consuming */

        pthread_cond_signal(&Buf.sigcon);

        /*sched_yield();*/

    }
}
```

Page 4:

```
/* The consumer thread */
void consumer(void)
{
    int flag=0;

    /* check to see if any buffers are filled or if the done flag is set */
    while (!flag) {

        /* lock the mutex */
        pthread_mutex_lock(&Buf.buflock);

        /* check to see if any buffers are filled */
        /* if not then wait for the condition to become true */
        while((Buf.occ <= 0)&&(Buf.done==0))
            pthread_cond_wait(&Buf.sigcon, &Buf.buflock);

        /* write the data from the buffer to the file */
        if((Buf.done==1) && (Buf.occ==0))
        {
            flag=1;
            printf("Consumer done!\n");
        }
        else
        {
            printf("Consumed Number %d: %d\n",Buf.tot,Buf.numb[Buf.occ]);
            Buf.tot++;

            /* decrement the number of buffers that are full */
            Buf.occ--;
        }

        /* release the mutex */
        pthread_mutex_unlock(&Buf.buflock);

        /* signal the producer that a buffer is empty */
        pthread_cond_signal(&Buf.sigpro);
    }
}
```

Some exercises to understand what is Pthread?